

Employee Management System (EMS)

Project Plan · Technical Document · 14-Day Build Schedule · Claude Code Prompts

Attendance · Projects · EOD Reports · Leave Calendar · Role Dashboards · Mail Notifications

Product	Internal Employee Management System (plug-and-play, Docker-based)
Roles	Admin (full control), MD (oversight + approvals), Employee (self-service)
Stack	React/Next.js 15 + FastAPI + PostgreSQL 16 + Redis 7 + Zoho Mail SMTP + Docker
Timeline	14 build days (D1–D14), one Claude Code prompt per day
Prepared	21 July 2026 · v2.0 FINAL (all features + machine-readable spec)

How to Read This Document (Human / AI / Machine)

- **Humans:** read Sections 1–7 for the plan, 9–10 for risks and done-criteria. Plain language, tables, no code required.
- **AI (Claude Code):** Section 8 contains one self-contained prompt per build day. Paste a prompt into a cleared session inside the repo; each prompt carries its own context, stack, non-negotiables, deliverables, guardrails, and verification — no other section needed.
- **Machines / agents:** Appendix A is the complete specification as strict JSON (roles, permissions, rules, tables, mail events, schedule). Extract the PDF text, take everything between the BEGIN and END marker lines (marker word: EMS_SPEC_JSON, wrapped in equals signs; the exact literals appear only in Appendix A), strip page footers, and parse. The same JSON is ALSO embedded inside this PDF as a file attachment named `ems_spec.json` — the most reliable machine path. The same JSON ships alongside this PDF as `ems_spec.json`.

1. Project Overview

EMS is an internal web application for a small-to-mid team where three roles operate on shared data with very different powers. Think of it like a factory time-office made digital: the punch clock (login/logout) is tamper-proof for workers, the floor manager (MD) sees every punch in real time via email, and the owner (Admin) holds the master keys and decides who gets which key and which mail.

Core principles

- **Immutability where it matters.** An employee's login/logout punch, once recorded, can never be edited by the employee. Every punch stores two timestamps: the **claimed time** (what the employee typed, e.g. 9:15) and the **server time** (when the form was actually submitted, e.g. 9:30). Both are saved and both are visible to MD/Admin.
- **Auto-marked calendar.** If an employee records no login on a working day, the calendar auto-marks that day (configurable label: Absent by default; Holiday if Admin marks the date as a company holiday). Working days, weekends, and holidays are Admin-configured.
- **Templated EOD.** End-of-Day reports start from an editable template (sections pre-filled) so employees modify rather than type from scratch.
- **Role-scoped dashboards.** Every role lands on its own dashboard: Employee (self only), MD (all employees, read + status edit), Admin (everything editable). Each dashboard also shows that user's own activity log.
- **Mail on every punch.** Each login and logout fires an email. **Who receives which mail is decided by Admin** via mail-routing rules (default: MD receives all punch mails).
- **Device fingerprint on every punch.** Each punch stores IP address, device class (mobile / tablet / desktop-class), operating system + version, and browser, parsed server-side from the user-agent. MD and Admin see these beside every punch. *Honest limit: a browser cannot distinguish a laptop from a desktop tower — both report as desktop-class.*
- **Full audit trail.** Every write action by every role (including Admin) is appended to an immutable audit log.
- **Plug and play.** One docker compose up brings up the whole stack with seeded Admin account and demo data. No manual DB setup.

2. Roles & Permission Matrix

Capability	Employee	MD	Admin
Login / Logout punch (own)	Yes (immutable after save)	Yes	Yes
Edit any punch record	No	No	Yes (logged in audit)

Capability	Employee	MD	Admin
View own dashboard, log, EOD, status	Yes	Yes	Yes
View ALL employees' logs & activity	No	Yes	Yes
See claimed time vs server time	Own only	Yes, all	Yes, all
See punch device (class, OS, browser, IP)	Own only	Yes, all	Yes, all
Submit EOD (from template)	Yes	View all	View/edit all
Project status update (assigned)	Yes	Yes + edit any	Yes + edit any
Create projects / assign work	No	Yes	Yes
Add new employee	No	Yes	Yes
Create roles / assign roles	No	No	Yes
Calendar: view	Yes	Yes	Yes
Calendar: mark holidays / working days	No	Yes	Yes
Leave: request	Yes	—	—
Leave: approve / reject	No	Yes	Yes
Receive punch emails	No	Default yes	Configurable
Configure mail routing (who gets what)	No	No	Yes
View full audit log (incl. Admin actions)	No	No	Yes
Announcements: post	No	Yes	Yes
Comments / team communication	Yes	Yes	Yes

3. Feature Set

3.1 Requested features

- Login / logout with immutable punch records and dual timestamps (claimed + server).
- Calendar: working days, weekends, holidays, leave, absent; auto-mark when no login recorded.
- EOD reports with Admin-managed templates (pre-filled sections, minimal typing).
- Project management: projects, tasks, assignment, status updates, MD/Admin edit rights.
- Per-role dashboards, each showing the user's own activity log inline.
- Email to MD on every login and logout; Admin decides mail routing per event type.
- Admin: role builder (create role, set permissions, assign members), full edit access, admin actions also logged.
- Communication: announcements + task/project comment threads.

3.2 Recommended additions (small cost, high value)

- **Leave request workflow** — employee applies, MD/Admin approves; approved leave shows on calendar (you asked for a leave calendar; this makes it self-serve).
- **Half-day and WFH markers** — punch form lets employee tag Half Day / Work From Home; visible on calendar.
- **IP + device capture on punch** — store IP address and user-agent with every punch (cheap fraud signal next to the dual timestamps).

- **Late-login flag** — Admin sets office start time (e.g. 9:30); punches after it are auto-flagged; monthly late count on MD dashboard.
- **Monthly attendance export** — CSV/Excel export of attendance, leave, and late counts for payroll.
- **In-app notification bell** — mirrors the emails inside the app (leave approved, task assigned, announcement posted).
- **Missed-EOD reminder** — if logged in but no EOD by a set hour, send reminder mail (routing Admin-controlled).
- **Nightly DB backup** — cron container dumping PostgreSQL to a mounted volume; reliability you asked for.

4. Tech Stack & Architecture

Layer	Choice	Why
Frontend	Next.js 15 (App Router), Tailwind	Same stack as Mattress.in / Agri.in — reuse patterns, components, deploy knowledge.
Backend	FastAPI (Python 3.13), Pydantic v2, SQLAlchemy ORM	Modular monolith, one schema per module, SecureRouter 401-by-default.
Database	PostgreSQL 16	UUIDv7 PKs, cursor pagination, Alembic migrations.
Cache / rate limit	Redis 7	Login rate limiting, session helpers, notification queue.
Mail	Zoho Mail SMTP (smtp.zoho.in) via aiosmtplib + background worker	Punch mails must never block the punch API; queued and retried. Zoho app-password auth.
Auth	JWT in httpOnly cookies + RBAC permission checks	Same security non-negotiables block you already use.
Packaging	Docker Compose (web, api, db, redis, mailworker, backup)	Plug and play: one command up, seeded admin, demo data.

Architecture in one picture (described)

Browser → Next.js (server components + API proxy) → FastAPI → PostgreSQL. FastAPI publishes mail events to a Redis queue; a mail worker consumes the queue, looks up Admin's routing rules, and sends via SMTP. The audit middleware sits inside FastAPI: every authenticated write is intercepted and appended to `audit.audit_log` before the response returns. Attendance immutability is enforced three times: no UPDATE/DELETE endpoint exists for employees, a DB trigger rejects UPDATES on punch rows unless the session role is admin, and the ORM model marks the rows read-only after insert.

5. Data Model (core tables)

Table	Key columns	Notes
auth.users	id (UUIDv7), email, password_hash, full_name, role_id, is_active	bcrypt + SHA-256 pre-hash (the fix you already shipped on Mattress.in).
auth.roles	id, name, is_system	Seeded: admin, md, employee. Admin can create more.
auth.role_permissions	role_id, permission_key	Permission keys map 1:1 to the matrix in Section 2.
attendance.punches	id, user_id, punch_type (in/out), claimed_at, server_at, ip, user_agent, device_type (mobile/tablet/desktop), os_name, os_version, browser, work_mode (office/wfh/half)	INSERT-only for non-admin. Dual timestamps + device fingerprint parsed server-side (ua-parser). One punch pair per day.
calendar.days	date, day_type (working/weekend/holiday), label	Admin/MD manage; feeds auto-marking.
calendar.day_status	user_id, date, status (present/absent/leave/holiday/half/wfh), source (auto/manual)	Nightly job writes auto rows; Admin can override (audited).
leave.requests	id, user_id, from_date, to_date, reason, status, decided_by	Approve/reject by MD or Admin; approved rows sync to day_status.
eod.templates	id, name, body_md, is_default	Admin-managed markdown templates with placeholder sections.

Table	Key columns	Notes
eod.reports	id, user_id, date, body_md, template_id, submitted_at	One per user per day; employee can edit until midnight, then locked.
projects.projects	id, name, code, status, created_by	Status: planned / active / on_hold / done.
projects.tasks	id, project_id, title, assignee_id, status, priority, due_date	Employee updates own task status; MD/Admin edit anything.
projects.status_updates	id, project_id, user_id, note, created_at	Timeline of status notes shown on project page.
comms.announcements	id, author_id, title, body, pinned	MD/Admin post; all read.
comms.comments	id, entity_type, entity_id, author_id, body	Threads on tasks and projects.
mail.routing_rules	id, event_type, recipient_role/user, enabled	Admin decides who gets login/logout/leave/EOD-miss mails.
mail.outbox	id, event_type, payload, status, attempts	Queue table; worker retries failures — punches never lost.
audit.audit_log	id, actor_id, actor_role, action, entity, entity_id, before, after, at	INSERT-only. Includes Admin actions. Admin can view, not edit.

6. Key Business Rules (non-negotiables)

- **R1 — Punch immutability:** employees have INSERT-only access to punches. No edit, no delete, ever. Admin edits go through a special endpoint that snapshots before/after into the audit log.
- **R2 — Dual timestamps:** `claimed_at` = time the employee states; `server_at` = DB clock at insert. UI for MD/Admin shows both side by side with the gap highlighted when > 10 minutes.
- **R2b — Device capture:** the punch API ignores any client-sent device info and derives everything itself: IP from the request (X-Forwarded-For aware behind nginx), device class / OS / browser parsed from the user-agent with the ua-parser library. Stored on the punch row, immutable like the rest.
- **R3 — Auto day-marking:** nightly job (00:30 IST) walks yesterday: no login punch on a working day → Absent (or Holiday if the calendar says so); approved leave → Leave; login exists → Present/Half/WFH from `work_mode`.
- **R4 — Mail on punch:** every punch inserts a row in mail.outbox in the same DB transaction; the worker sends per Admin routing rules. Punch API responds fast even if SMTP is down.
- **R5 — EOD lock:** EOD editable by its author only until 23:59 of its date; after that read-only (Admin can still edit, audited).
- **R6 — RBAC from DB, not code:** permissions loaded from `role_permissions` so Admin's custom roles work without deploys.
- **R7 — Everything audited:** all writes by all roles land in `audit_log`, including Admin. Audit log is INSERT-only at DB level.
- **R8 — One command up:** `docker compose up -d` must yield a working app with seeded Admin (admin@company.local / printed one-time password), default roles, default EOD template, current-year calendar, and demo employees.

7. Day-wise Build Schedule (D1–D14)

Day	Deliverable	Verify by
D1	Monorepo scaffold, Docker Compose (web/api/db/redis), Alembic baseline, health checks, CI lint	compose up → /health 200, web hello page
D2	Auth: register-by-admin, login, logout, JWT httpOnly, bcrypt+SHA-256, Redis rate limit, seeded roles & admin	Login as seeded admin; wrong password 5x → 429
D3	RBAC engine: roles, permissions, role builder API, SecureRouter guards, audit middleware v1	Create custom role via API; employee blocked from admin route; writes appear in <code>audit_log</code>
D4	Attendance: punch in/out, dual timestamps, IP/UA capture, <code>work_mode</code> , immutability (API + DB trigger)	Punch, attempt edit as employee → 403; DB UPDATE as app role → rejected
D5	Calendar engine: day types, holiday CRUD, nightly auto-marker, month view API	Mark holiday; run marker job; <code>day_status</code> rows correct
D6	Leave module: request, approve/reject, sync to calendar, balances (optional simple counter)	Employee applies → MD approves → calendar shows Leave
D7	EOD module: templates CRUD (admin), report submit from template, midnight lock	Submit EOD from template; edit after lock → 403
D8	Projects: projects + tasks CRUD, assignment, status updates timeline	MD creates project, assigns task; employee updates status
D9	Mail engine: outbox, worker, SMTP, Admin routing rules UI/API, punch & leave & EOD-miss events	Punch → MD inbox (MailHog in dev); flip routing → mail goes elsewhere
D10	Employee dashboard: punch card, my calendar, my EOD, my tasks, my activity log	All widgets live with real data

Day	Deliverable	Verify by
D11	MD dashboard: team punch feed (claimed vs server), employee drill-down, project status board, EOD reader, late flags	Gap >10 min highlighted; drill into one employee's full activity
D12	Admin dashboard: user & role management, calendar admin, mail routing, template manager, full audit viewer, edit-anything flows	Admin edits a punch → before/after visible in audit log
D13	Communication: announcements, task/project comments, notification bell, exports (attendance CSV)	Post announcement → bell for all; export downloads
D14	Hardening & ship: security headers, backups container, seed/demo data, README runbook, Lighthouse pass, VPS deploy notes	Fresh clone → compose up → full demo in <5 min

Sequencing logic: auth and RBAC first because every later module hangs permissions off them; attendance before calendar because auto-marking reads punches; mail engine after the modules that emit events; dashboards after all data exists; hardening last. Same Vertical Slice discipline as Agri.in — each day lands migration + API + minimal UI + tests for that slice.

8. Claude Code Prompts (one per day)

Paste each prompt into a cleared Claude Code session inside the repo. House style: context → reuse → stack → non-negotiables → lettered deliverables → guardrails → verification. Keep CLAUDE.md updated at the end of every day (each prompt tells Claude Code to do so).

Day 1 — Scaffold & Infrastructure

CONTEXT: New project "EMS" - internal Employee Management System for Oneuni. Roles: admin, md, employee. 14-day build. Today is Day 1: repo scaffold and plug-and-play infrastructure only.

REUSE: Follow the same monorepo conventions as the mattress repo: /apps/web (Next.js 15 App Router, Tailwind), /apps/api (FastAPI, Python 3.13), /infra (docker), pnpm 9 workspaces, ruff + eslint configs copied from house defaults.

STACK: Next.js 15, FastAPI, PostgreSQL 16, Redis 7, Docker Compose, Alembic.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against auth.role_permissions. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

- A. Monorepo scaffold with pnpm workspaces; /apps/web hello page; /apps/api with /health returning {status:"ok", version}.
- B. docker-compose.yml: web(3000), api(8000), db(5432, volume), redis(6379), mailhog(8025 UI) for dev mail. Healthchecks on all services; api waits for db.
- C. Alembic initialized with baseline empty migration; make target "make migrate".
- D. .env.example with every variable documented; config loader with pydantic-settings; ENVIRONMENT=dev default.
- E. CLAUDE.md created at repo root: stack, conventions, non-negotiables block above, day log started.
- F. GitHub Actions: lint + typecheck on push.

SCOPE GUARDRAILS: No auth, no models, no business logic today. Do not add extra services.

VERIFY: docker compose up -d from fresh clone -> web shows hello page on :3000, GET :8000/health returns 200, mailhog UI loads on :8025, CI green.

Day 2 — Auth Foundation

CONTEXT: EMS Day 2. Scaffold from D1 is live. Today: authentication and seeded roles. No RBAC engine yet (that is D3) - just the three system roles hardcoded in seed.

REUSE: Copy the auth patterns from Mattress.in: JWT httpOnly cookie flow, SHA-256 pre-hash + direct bcrypt (do NOT use passlib - known 4.x incompatibility), Redis-backed rate limiter.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against auth.role_permissions. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

- A. Schema auth: users, roles, role_permissions tables (migration). Seed roles: admin, md, employee (is_system=true). role_permissions left empty today.
- B. Seed script: creates admin@company.local with a random one-time password PRINTED ONCE to container logs on first boot only; idempotent on restart.
- C. Endpoints: POST /auth/login, POST /auth/logout, GET /auth/me. Login sets httpOnly cookie; logout clears it. Rate limit: 5 fails / 15 min per IP+email -> 429.
- D. POST /users (admin-only via temporary role check on user.role name) to create md/employee accounts with forced password reset flag.
- E. Web: /login page, auth context, protected route wrapper redirecting to /login; /me debug page showing current user + role.
- F. Tests: login happy path, wrong password, rate limit trip, cookie flags asserted. Update CLAUDE.md day log.

SCOPE GUARDRAILS: No permission matrix, no dashboards, no attendance. No password-reset email flow yet (flag only).

VERIFY: Seeded admin logs in on :3000/login, /auth/me returns role admin, 6th bad login returns 429, cookies are httpOnly+SameSite=Lax.

Day 3 — RBAC Engine + Audit Middleware

CONTEXT: EMS Day 3. Auth works. Today: database-driven RBAC (Admin can create custom roles without deploys) and the audit middleware every later module depends on.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against auth.role_permissions. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

- A. Permission registry: single source-of-truth python dict of permission keys (attendance.punch, attendance.view_all, eod.submit, eod.view_all, projects.manage, projects.update_status, calendar.manage, leave.approve, users.manage, roles.manage, mail.route, audit.view, comms.announce ... full list from spec Section 2). Migration seeds role_permissions: admin=all, md=oversight set, employee=self-service set.
- B. SecureRouter upgraded: @requires("permission.key") decorator reads the user's role permissions (cached in Redis 60s, invalidated on role edit).
- C. Role management API (roles.manage): create role, clone role, set permissions, assign user to role. System roles cannot be deleted or renamed.
- D. Schema audit: audit_log table INSERT-only (DB trigger blocks UPDATE/DELETE for app db user). Middleware: every authenticated non-GET request appends actor, role, action, entity, entity_id, before/after JSON snapshots.
- E. Tests: employee hitting admin route -> 403; custom role with one permission works; audit rows written with correct before/after; audit UPDATE attempt at DB level fails.
- F. Update CLAUDE.md: permission key list is now canonical - future days must register new keys here.

SCOPE GUARDRAILS: No UI for role management today (Admin dashboard D12 covers it). API + tests only.

VERIFY: Create role "intern" with only attendance.punch via API as admin; login as intern-assigned user -> punch route authorized, everything else 403; all of it visible in audit_log.

Day 4 — Attendance Engine (Immutable Punches)

CONTEXT: EMS Day 4. RBAC + audit live. Today is the heart of the product: tamper-proof login/logout punches with dual timestamps.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against auth.role_permissions. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

- A. Schema attendance: punches(id, user_id, punch_type in|out, claimed_at, server_at default now() set by DB, ip, user_agent, device_type mobile|tablet|desktop, os_name, os_version, browser, work_mode office|wfh|half, day date generated from server_at IST). Partial unique index: max one 'in' and one 'out' per user per day.
- B. Immutability, enforced three ways: (1) no UPDATE/DELETE endpoints for punches except admin-only PATCH /admin/punches/{id}; (2) DB trigger rejects UPDATE/DELETE unless session var app.role='admin' is set (set only inside the admin endpoint transaction); (3) ORM model flagged read-only post-insert.
- C. Endpoints: POST /attendance/punch {punch_type, claimed_at, work_mode} - server captures ip (X-Forwarded-For aware) + user-agent itself and parses device_type/os_name/os_version/browser via the ua-parser library (pip: ua-parser). NEVER trust client-sent device fields - derive server-side only. Note in code comment: laptop vs desktop tower is not distinguishable from UA; both stored as desktop; GET /attendance/me?month=; GET /attendance/users/{id} (attendance.view_all); admin PATCH with mandatory reason field, full before/after audit.
- D. Business checks: cannot punch out without punch in; claimed_at must be same calendar day; claimed/server gap computed and returned.

E. Web: punch widget component (big In/Out button, time field prefilled with now, work-mode selector, disabled state after punch) - wired on a temp /punch page until dashboards exist.
F. Tests: dual timestamps saved; UA string 'iPhone Safari' parses to mobile/iOS, 'Windows Chrome' to desktop/Windows; employee PATCH -> 403; raw SQL UPDATE as app user -> trigger error; unique per day enforced; admin edit lands in audit with reason.

SCOPE GUARDRAILS: No mail yet (D9). No calendar marking yet (D5). No dashboard placement.

VERIFY: Punch in with claimed 09:15 while server clock says 09:30 from a phone -> both times stored plus device=mobile, os=Android/iOS, browser, ip; GET as MD shows gap 15m and device columns; employee cannot alter the row by any path.

Day 5 — Calendar Engine + Auto Day-Marking

CONTEXT: EMS Day 5. Punches exist. Today: company calendar (working days / weekends / holidays) and the nightly job that auto-marks each employee's day - including the rule: no login recorded on that day => day auto-marked (Absent on working day, Holiday label if the date is a company holiday).

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against auth.role_permissions. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

- A. Schema calendar: days(date PK, day_type working|weekend|holiday, label) and day_status(user_id, date, status present|absent|leave|holiday|half|wfh, source auto|manual, marked_at). Seed current year: Mon-Fri working, Sat-Sun weekend, Indian national holidays + TN state holidays as company holidays (editable later).
- B. Calendar management API (calendar.manage): upsert day type/label, bulk range ops (e.g. mark 2nd Saturdays as holiday).
- C. Auto-marker job: idempotent function mark_day(date) - for each active user: approved leave -> leave; company holiday -> holiday; no 'in' punch on working day -> absent; punch exists -> present/half/wfh from work_mode. Scheduled 00:30 IST daily for yesterday via APScheduler in api container; also exposed as admin-only POST /admin/jobs/mark-day for backfill.
- D. Month view API: GET /calendar/me?month= and /calendar/users/{id}?month= merging day types + statuses; GET /calendar/team?month= (attendance.view_all) returns per-user grid.
- E. Web: reusable <MonthCalendar> component with color legend (present green, absent red, leave amber, holiday grey, wfh blue, half striped) on temp /calendar page.
- F. Tests: marker idempotency; holiday overrides absent; manual admin override wins over auto and is audited.

SCOPE GUARDRAILS: Leave requests are D6 - marker just reads an (empty for now) approved-leave lookup interface. No dashboards.

VERIFY: Backfill last week via admin job; a day with no punch on a working day shows Absent; declare it a holiday and re-run -> flips to Holiday.

Day 6 — Leave Management

CONTEXT: EMS Day 6. Calendar live. Today: leave request/approval workflow feeding the calendar.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against auth.role_permissions. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

- A. Schema leave: requests(id, user_id, from_date, to_date, half_day bool, reason, status pending|approved|rejected|cancelled, decided_by, decided_at, decision_note). Simple balances table (user_id, year, allotted, used) seeded 18/year default; Admin-editable.
- B. Endpoints: employee POST /leave (validations: no overlap with existing pending/approved, dates >= today unless leave.backdate permission); GET /leave/me; DELETE own pending. MD/Admin (leave.approve): GET /leave/pending, POST /leave/{id}/approve|reject with note. Approval inside one transaction: update request, increment used balance, write day_status leave rows (source manual), audit.
- C. Hook the D5 marker's approved-leave lookup to this table for real.

D. Web: /leave temp page - employee form + my-requests list with status chips; approver queue view with approve/reject inline.
E. Tests: overlap blocked; approve syncs calendar + balance; reject leaves calendar untouched; cancel returns balance.

SCOPE GUARDRAILS: No email yet (D9 wires leave events into routing). No multi-level approval chains.

VERIFY: Employee applies 2 days -> MD approves -> both dates show Leave on employee calendar and balance used +2.

Day 7 — EOD Reports with Templates

CONTEXT: EMS Day 7. Today: End-of-Day reports. Key requirement: EOD has admin-managed templates so employees edit/modify instead of typing from scratch, and reports lock at midnight.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against `auth.role_permissions`. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

- A. Schema eod: templates(id, name, body_md, is_default, active) and reports(id, user_id, date, template_id, body_md, submitted_at, locked bool). Unique (user_id, date). Seed default template: `### Today's Work\n- \n### Completed\n- \n### In Progress\n- \n### Blockers\n- None\n### Tomorrow's Plan\n-`
- B. Template CRUD (eod.templates permission, admin): create/edit/deactivate; exactly one default enforced.
- C. Report endpoints: GET /eod/template/default; POST /eod (date=today only, body prefilled client-side from template); PATCH own report until 23:59 IST of its date, then locked=true via the nightly job (extend D5 scheduler); GET /eod/me?month=; GET /eod/users/{id}?date= and GET /eod/team?date= (eod.view_all). Admin PATCH any (audited).
- D. Web: /eod temp page - markdown editor prefilled with template, autosave draft to localStorage-free (use API draft status), preview toggle; MD reader view listing team EODs by date with submitted/missing badges.
- E. Missing-EOD detector: function returning users who punched in but have no EOD for a date (used by D9 reminder mail and D11 dashboard).
- F. Tests: template prefill; edit after lock -> 403; unique per day; admin edit audited.

SCOPE GUARDRAILS: No mail. No rich-text - markdown only.

VERIFY: Submit EOD from template in under 30 seconds of edits; next-day edit attempt fails; MD view shows who is missing.

Day 8 — Project Management

CONTEXT: EMS Day 8. Today: projects, tasks, assignment, and the status-update timeline. Employees update status on assigned work; MD/Admin can create projects, add work, and edit any status.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against `auth.role_permissions`. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

- A. Schema projects: projects(id, name, code unique, description, status planned|active|on_hold|done, created_by), tasks(id, project_id, title, description, assignee_id, status todo|in_progress|review|done, priority low|med|high, due_date), status_updates(id, project_id, task_id nullable, user_id, note, old_status, new_status, created_at).
- B. Endpoints: projects CRUD (projects.manage - MD/Admin); tasks CRUD (projects.manage) + assignee-only PATCH /tasks/{id}/status (projects.update_status) which also inserts a status_updates row; GET /projects list with task counts by status; GET /projects/{id} with tasks + timeline; GET /tasks/me.
- C. Rule: employees may update status and add notes ONLY on tasks assigned to them (ownership check); MD/Admin edit anything (audited).
- D. Web: /projects temp pages - project list, project detail with kanban-lite columns (no drag needed - status dropdown fine), my-tasks list, timeline feed on project page.

E. Tests: non-assignee status change -> 403; timeline row on every status change; MD edit any task ok.

SCOPE GUARDRAILS: No comments (D13). No file attachments. No Gantt.

VERIFY: MD creates project + assigns 3 tasks; employee moves one to in_progress with a note; timeline shows old->new status with author and time.

Day 9 — Mail Engine + Admin Routing Rules

CONTEXT: EMS Day 9. All event sources exist. Today: the mail system. Hard requirements: every login and logout sends a mail (default recipient: MD); WHO receives WHICH mail is decided by Admin; punch API must never block on SMTP.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against auth.role_permissions. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

A. Schema mail: routing_rules(id, event_type, recipient_type role|user|email, recipient_value, enabled) and outbox(id, event_type, payload jsonb, to_emails text[], status pending|sent|failed, attempts, last_error, created_at, sent_at). Seed rules: punch.in -> role:md, punch.out -> role:md, leave.requested -> role:md, leave.decided -> user:requester, eod.missing -> role:md.

B. Event emit: same-transaction outbox insert from punch (retrofit D4), leave (D6), and a 20:30 IST scheduled eod.missing check (D7 detector). Recipient resolution happens in the worker at send time using current routing rules.

C. Worker: separate container "mailworker" polling outbox (SELECT ... FOR UPDATE SKIP LOCKED), renders template per event type, sends via aiosmtplib. Production: Zoho Mail - SMTP_HOST=smtp.zoho.in, SMTP_PORT=465 (SSL) or 587 (STARTTLS), SMTP_USER=full mailbox address, SMTP_PASS=Zoho app-specific password (document in .env.example: generate under Zoho Account > Security > App Passwords; normal password will NOT work with 2FA). From-address must equal the authenticated Zoho mailbox or an approved alias, else Zoho rejects. Dev: mailhog. Exponential backoff, max 5 attempts then failed.

D. Punch mail template must show: employee name, punch type, CLAIMED time and SERVER time side by side with the gap, work mode, and the device line: device class + OS + browser + IP (e.g. Mobile - Android 14 - Chrome - 103.x.x.x).

E. Routing API (mail.route, admin): CRUD rules, test-send endpoint, outbox viewer with retry-failed action.

F. Tests: punch inserts outbox row in same txn (rollback removes it); worker sends to mailhog; disabling a rule stops sends; failed retry works.

SCOPE GUARDRAILS: No HTML template designer - one clean base template. No digest/batching.

VERIFY: Punch in -> mail visible in MailHog addressed to MD within 5s showing 09:15 claimed / 09:30 server (gap 15m); Admin reroutes punch.in to a custom email -> next punch goes there.

Day 10 — Employee Dashboard

CONTEXT: EMS Day 10. Backend complete. Dashboards start. Today: the employee's home. Requirement: employee sees own activities, log, EOD, project status, login/logout status - all in one place; dashboard also displays their own activity log.

REUSE: <MonthCalendar>, punch widget, EOD editor, my-tasks list from earlier temp pages - compose, don't rebuild. Delete temp pages after migrating.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against auth.role_permissions. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

A. /dashboard route (role employee) with widgets: (1) Punch card - current status (in since 09:15 / out), today's pair, In/Out action; (2) My Calendar month with legend + leave-balance chip + "Apply Leave" opening the D6 form in a modal; (3) Today's EOD - template-prefilled editor inline, submitted state after; (4) My Tasks grouped by status with quick status dropdown; (5) My Activity Log - reverse-chron feed of my punches, EOD submits, task updates, leave events (reads audit_log filtered to

actor=me, paginated by cursor).

- B. Header: name, role badge, notification bell placeholder (D13), logout.
- C. Empty states and loading skeletons for every widget; mobile-responsive single column.
- D. Post-login redirect by role: employee -> /dashboard, md -> /md (D11), admin -> /admin (D12) - stub the latter two.
- E. Tests: employee sees only own data; direct API probing for others' data still 403 (re-assert).

SCOPE GUARDRAILS: No MD/Admin screens. No theming system - Tailwind house style.

VERIFY: Login as demo employee: punch in, submit EOD from template, move a task, apply leave - all without leaving /dashboard; activity log shows all four actions.

Day 11 — MD Dashboard

CONTEXT: EMS Day 11. Today: MD's oversight home. Requirements: view all employee logs and full activity, see claimed vs server punch times, view/edit project status, add work, read EODs, calendar access, add new employee.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against `auth.role_permissions`. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

- A. /md route (permission-gated, not role-name-gated - custom roles with the same permissions must work): (1) Live Punch Feed - today's punches, columns claimed | server | gap | device | OS | browser | IP, gap>10m highlighted coral, device rendered with an icon (phone/tablet/monitor) + tooltip showing full user-agent; (2) Team Attendance - month grid (rows=employees, cols=days) from /calendar/team with status colors and late-login count per employee (office start time from settings, default 09:30); (3) EOD Reader - date picker, submitted list + missing list, full report view; (4) Projects Board - all projects with status editing, add-task (assign work) inline; (5) Employee Drill-down - /md/employees/{id}: their calendar, punches with dual times AND device/OS/browser/IP per punch (flag when an employee punches from a new device or new IP vs their history), EOD history, tasks, full activity feed; (6) Add Employee - form creating user (users.manage) with role picker limited to non-admin roles.
- B. Leave approvals queue widget (D6 approver view) on the dashboard.
- C. MD's own punch card + own activity log section (MD also punches; requirement says every dashboard shows own log).
- D. Settings read: `office_start_time` from a small settings table (admin-editable D12); create it now with defaults.
- E. Tests: MD edits project status ok; MD cannot open /admin routes; late flag math correct.

SCOPE GUARDRAILS: No role management, no mail routing, no audit-log-of-others UI (Admin-only). No CSV export yet (D13).

VERIFY: With demo data: feed shows an employee whose claimed 09:15 / server 09:30 punch is highlighted; MD approves a leave, edits a project status, adds a task, opens the employee drill-down and sees everything.

Day 12 — Admin Dashboard (Everything Editable)

CONTEXT: EMS Day 12. Today: Admin's control room. Requirements: all access, everything editable, create roles and members, assign roles, decide mail routing, view ALL logs including admin's own actions.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against `auth.role_permissions`. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

- A. /admin route with sections: (1) Users - full CRUD, activate/deactivate, force password reset, assign role; (2) Roles - role builder UI over D3 API: create/clone role, permission checkbox matrix grouped by module, member assignment; system roles locked; (3) Calendar Admin - year view, click/range to set working/weekend/holiday with label, bulk tools; (4) Attendance Admin - search punches, admin-edit with mandatory reason (D4 endpoint), edited rows badged "admin-edited" everywhere they render; (5) EOD Templates - manager over D7 CRUD with live preview; (6) Mail Routing - rules table over D9 API + outbox

viewer with retry; (7) Settings - office_start_time, EOD lock time, timezone; (8) Audit Log Viewer - filter by actor/entity/action/date, cursor-paginated, before/after JSON diff viewer; explicitly includes admin actions; read-only.

B. Admin dashboard also shows admin's own punch card and own activity log (consistency rule).

C. Danger guards: destructive actions need typed confirmation; deactivating last admin blocked.

D. Tests: admin punch-edit shows in audit with before/after; role created in UI takes effect without restart (Redis cache invalidation from D3 verified end-to-end).

SCOPE GUARDRAILS: No comms/exports (D13). No backup UI (D14 is infra-level).

VERIFY: Admin: creates role "HR" with users.manage + leave.approve only, creates a user on it, logs in as that user in incognito -> sees exactly those powers; admin edits a punch -> audit diff renders; audit list shows the admin's own edit.

Day 13 — Communication, Notifications & Exports

CONTEXT: EMS Day 13. Today: the communication layer and quality-of-life features - announcements, comments, in-app notification bell, attendance export.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against auth.role_permissions. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

A. Schema comms: announcements(id, author_id, title, body_md, pinned, created_at) and comments(id, entity_type project|task, entity_id, author_id, body, created_at). notifications(id, user_id, type, payload, read_at).

B. Announcements: post/edit/pin (comms.announce - MD/Admin); all users see latest on their dashboard (new widget on all three dashboards) + /announcements archive page.

C. Comments: threads on project and task detail pages; authors edit own within 15 min; MD/Admin moderate (delete, audited).

D. Notification bell: fan-out notification rows on announcement posted, task assigned, leave decided, comment on your task; header bell with unread count on ALL dashboards, mark-read; polling every 30s (no websockets).

E. Exports (attendance.view_all): GET /export/attendance.csv?month=&user_id= - rows: user, date, status, in-claimed, in-server, out-claimed, out-server, gap-minutes, work mode, late flag, device, os, ip. Streamed response. Button on MD and Admin dashboards.

F. Tests: employee cannot announce; bell count correct; CSV columns exact and IST times.

SCOPE GUARDRAILS: No DMS/chat rooms. No push/email for comments (mail routing stays punch/leave/EOD only).

VERIFY: MD posts pinned announcement -> appears on employee dashboard + bell rings; comment on a task notifies assignee; export of last month opens clean in Excel.

Day 14 — Hardening, Seed, Docs & Ship

CONTEXT: EMS Day 14, final. Today: production hardening, plug-and-play polish, demo seed, docs, deploy notes. Target: fresh clone to full working demo in under 5 minutes with one command.

NON-NEGOTIABLES: FastAPI modular monolith, one PG schema per module. SecureRouter: every route 401-by-default, explicit permission key per endpoint checked against auth.role_permissions. UUIDv7 PKs everywhere. Cursor pagination only (offset banned by lint). Pydantic v2 models, SQLAlchemy ORM only (no raw SQL except migrations/triggers). JWT in httpOnly Secure SameSite=Lax cookies. Passwords: SHA-256 pre-hash then bcrypt (passlib banned). Redis rate limit on auth routes. All write endpoints wrapped by audit middleware once it exists (D3+). Ownership checks on every user-scoped resource.

DELIVERABLES:

A. Security pass: security headers middleware (CSP, X-Frame-Options DENY, nosniff, referrer-policy), CORS locked to configured origin, cookie Secure in production, dependency audit (pip-audit, pnpm audit) fixed or documented, rate limits re-checked on auth + punch.

B. Backup container: nightly pg_dump to ./backups volume, 14-day retention, restore script + tested restore doc.

C. Seed/demo mode (SEED_DEMO=true): 1 admin, 1 MD, 6 employees, 30 days of realistic punches (some late, some gaps, one WFH, one half-day), holidays, 2 projects with tasks and timelines, EODs, a pending leave, sample announcement - so every dashboard is alive on first boot.

D. docker-compose.prod.yml: no mailhog, Zoho SMTP env (smtp.zoho.in:465 SSL, app password), restart

policies, resource limits; nginx site file + certbot notes for VPS deploy (Hostinger KVM4 pattern: /opt/ems, unique port block 3200/8200 to avoid mattress 3000/8000 and agri 3100/8100 collisions).
E. Docs: README (quickstart, env table, port map, backup/restore, troubleshooting), RUNBOOK.md (deploy steps, rollback), final CLAUDE.md cleanup.
F. Quality gates: Lighthouse ≥ 90 on dashboards over throttled 3G; full test suite green; e2e smoke script (login each role, punch, EOD, project update, mail assert against mailhog) runnable via make smoke.

SCOPE GUARDRAILS: No new features. Fix, polish, document only.

VERIFY: On a clean machine: git clone -> cp .env.example .env -> docker compose up -d -> :3000 login as each seeded role -> every dashboard populated, punch mail lands in mailhog, make smoke passes.

9. Risks & Operating Notes

- **Clock trust:** claimed_at is user input; server_at is the truth anchor. Never compute payroll from claimed_at alone.
- **SMTP reliability:** punches must not depend on mail. The outbox pattern (D9) guarantees mails survive SMTP outages; monitor the failed count on the Admin outbox viewer.
- **Timezone:** store UTC in DB, render IST everywhere, and run all day-boundary jobs (marker, EOD lock, missing-EOD) on IST boundaries. Off-by-one dates are the #1 bug class in attendance systems.
- **Custom roles vs UI:** gate screens by permission keys, never by role name, or Admin-created roles will see broken UIs.
- **Solo-dev pacing:** D9 (mail) and D12 (admin) are the heavy days; if a day overruns, push D13 features before touching D14 hardening — ship reliability over comfort features.
- **Device detection honesty:** device class/OS/browser come from the user-agent, which a technical user can spoof. Treat it as a strong signal beside IP and the claimed/server gap, not as proof. Laptop vs desktop tower is not detectable from a browser — both show as desktop-class.
- **Zoho limits:** Zoho Mail free/standard tiers have daily send caps; with per-punch mails for a growing team, monitor the outbox failed count and consider a punch digest mail as a future toggle.
- **VPS ports:** reserve 3200 (web) / 8200 (api) now so EMS can live beside Mattress (3000/8000) and Agri (3100/8100) on the same KVM4 with -p ems compose project naming.

10. Definition of Done (project level)

- One command boot with seeded demo; all three dashboards functional on first run.
- Employee cannot alter any punch by UI, API, or direct SQL as the app user.
- Every punch mail shows claimed and server time; Admin can reroute any mail without a deploy.
- A day with no login on a working day is auto-marked; holidays override.
- EOD from template in under a minute; locks at midnight.
- Every write by every role, including Admin, is in the audit log.
- Lighthouse ≥ 90 , tests green, backup + restore proven once.

Appendix A — Machine-Readable Specification (JSON)

The full project specification as strict JSON. Two machine paths: (1) BEST — this PDF carries an embedded file attachment named `ems_spec.json` (pristine JSON, extract with any PDF library); (2) text path — extract PDF text, take the block between the BEGIN/END marker lines below, remove the page-footer lines, remove all newline characters, then parse (visual line-wrapping may merge words inside long values; structure stays valid). The identical file also ships beside this PDF.

```
===EMS_SPEC_JSON_BEGIN===
{
  "project": {
    "name": "EMS - Employee Management System",
    "version": "2.0-final",
    "owner": "Oneuni Agri Platform Pvt Ltd",
    "prepared": "2026-07-21",
    "timezone": "Asia/Kolkata",
    "build_days": 14,
    "ports": {
      "web": 3200,
      "api": 8200,
      "reserved_reason": "avoid mattress 3000/8000 and agri 3100/8100 on shared KVM4"
    }
  },
  "stack": {
    "frontend": "React / Next.js 15 (App Router) + Tailwind",
    "backend": "FastAPI (Python 3.13), Pydantic v2, SQLAlchemy ORM, Alembic",
    "database": "PostgreSQL 16 (UUIDv7 PKs, cursor pagination only)",
    "cache": "Redis 7 (rate limiting, permission cache, notification polling)",
    "mail": {
      "provider": "Zoho Mail",
      "host": "smtp.zoho.in",
      "port": 465,
      "security": "SSL",
      "auth": "app-specific password",
      "dev": "mailhog"
    },
    "packaging": "Docker Compose (web, api, db, redis, mailworker, backup; mailhog dev-only)"
  },
  "roles": {
    "employee": {
      "permissions": [
        "attendance.punch",
        "attendance.view_own",
        "eod.submit",
        "eod.view_own",
        "projects.update_status_assigned",
        "projects.view_assigned",
        "calendar.view",
        "leave.request",
        "comms.comment",
        "dashboard.self"
      ]
    },
    "md": {
      "inherits": "employee",
      "permissions": [
        "attendance.view_all",
        "attendance.view_device_all",
        "eod.view_all",
        "projects.manage",
        "projects.edit_any_status",
        "users.create_non_admin",
        "calendar.manage",
        "leave.approve",
        "comms.announce",
        "export.attendance",
        "mail.receive_punch_default"
      ]
    }
  },
}
```

```

"admin": {
  "permissions": [
    "*"
  ],
  "extras": [
    "roles.manage",
    "mail.route",
    "audit.view",
    "attendance.edit_with_reason",
    "eod.templates",
    "settings.manage",
    "users.manage_all"
  ],
  "note": "admin actions are themselves audit-logged; admin can create custom roles from the permission registry without redeploy"
},
"business_rules": {
  "R1_punch_immutability": "Employees INSERT-only on punches. Enforced 3x: no edit endpoint, DB trigger blocking UPDATE/DELETE unless session app.role=admin, ORM read-only post-insert. Admin edits require reason and are audited with before/after.",
  "R2_dual_timestamps": "claimed_at = employee-stated time; server_at = DB clock at insert. Both stored, both shown to MD/Admin, gap highlighted when > 10 minutes.",
  "R2b_device_capture": "Server derives ip (X-Forwarded-For aware), device_type mobile|tablet|desktop, os_name, os_version, browser from user-agent via ua-parser. Client-sent device fields ignored. Known limit: laptop vs desktop tower indistinguishable; both = desktop.",
  "R3_auto_day_marking": "Nightly 00:30 IST for yesterday: approved leave->leave; company holiday->holiday; no 'in' punch on working day->absent; else present/half/wfh from work_mode. Idempotent; admin backfill endpoint.",
  "R4_mail_on_punch": "Every punch inserts mail.outbox row in same DB transaction; worker resolves recipients from admin routing rules at send time; default recipient role:md for punch.in and punch.out. SMTP outage never blocks or loses a punch.",
  "R5_eod_lock": "EOD editable by author until 23:59 IST of its date, then locked. Admin may still edit (audited).",
  "R6_rbac_from_db": "Permissions load from auth.role_permissions (Redis-cached 60s, invalidated on edit) so admin-created roles work without deploys. UI gates by permission key, never role name.",
  "R7_everything_audited": "All writes by all roles including admin -> audit.audit_log (INSERT-only at DB level) with actor, action, entity, before/after JSON.",
  "R8_plug_and_play": "docker compose up -d from fresh clone yields working app: seeded admin (one-time password printed once), default roles, default EOD template, current-year calendar, demo data when SEED_DEMO=true."
},
"features": {
  "requested": [
    "login/logout punches, immutable, dual timestamps",
    "calendar: working/weekend/holiday/leave/absent with auto-marking on missing login",
    "EOD reports from admin-managed templates (minimal typing), midnight lock",
    "project management: projects, tasks, assignment, status updates; MD/Admin edit any",
    "per-role dashboards (employee/MD/admin), each showing own activity log",
    "email on every login/logout; admin decides routing (who receives which mail)",
    "MD: add employee, view all logs/activity, claimed-vs-server times, device/OS/IP per punch, read EODs, calendar access",
    "admin: full access, role builder, assign roles, mail routing, full audit incl. admin's own actions",
    "communication: announcements + task/project comments"
  ],
  "added": [
    "leave request workflow with balances (default 18/yr) syncing to calendar",
    "half-day and WFH markers at punch time",
    "late-login flag vs admin-set office start (default 09:30) + monthly late counts",
    "monthly attendance CSV export incl. both timestamps, gap, device, os, ip",
    "in-app notification bell (leave decided, task assigned, announcement, comment)",
    "missed-EOD reminder mail at 20:30 IST (admin-routable)",
    "nightly pg_dump backup container, 14-day retention, tested restore",
    "new-device / new-IP flag on MD employee drill-down"
  ]
},
"data_model": {
  "auth.users": [

```

```

        "id uuidv7",
        "email",
        "password_hash sha256+bcrypt",
        "full_name",
        "role_id",
        "is_active"
    ],
    "auth.roles": [
        "id",
        "name",
        "is_system"
    ],
    "auth.role_permissions": [
        "role_id",
        "permission_key"
    ],
    "attendance.punches": [
        "id",
        "user_id",
        "punch_type in|out",
        "claimed_at",
        "server_at db-default",
        "ip",
        "user_agent",
        "device_type mobile|tablet|desktop",
        "os_name",
        "os_version",
        "browser",
        "work_mode office|wfh|half",
        "day generated IST",
        "UNIQUE(user_id, day, punch_type)"
    ],
    "calendar.days": [
        "date pk",
        "day_type working|weekend|holiday",
        "label"
    ],
    "calendar.day_status": [
        "user_id",
        "date",
        "status present|absent|leave|holiday|half|wfh",
        "source auto|manual",
        "marked_at"
    ],
    "leave.requests": [
        "id",
        "user_id",
        "from_date",
        "to_date",
        "half_day",
        "reason",
        "status pending|approved|rejected|cancelled",
        "decided_by",
        "decided_at",
        "decision_note"
    ],
    "leave.balances": [
        "user_id",
        "year",
        "allotted",
        "used"
    ],
    "eod.templates": [
        "id",
        "name",
        "body_md",
        "is_default",
        "active"
    ],
    ],

```

```

"eod.reports": [
  "id",
  "user_id",
  "date",
  "template_id",
  "body_md",
  "submitted_at",
  "locked",
  "UNIQUE(user_id,date)"
],
"projects.projects": [
  "id",
  "name",
  "code unique",
  "description",
  "status planned|active|on_hold|done",
  "created_by"
],
"projects.tasks": [
  "id",
  "project_id",
  "title",
  "description",
  "assignee_id",
  "status todo|in_progress|review|done",
  "priority low|med|high",
  "due_date"
],
"projects.status_updates": [
  "id",
  "project_id",
  "task_id?",
  "user_id",
  "note",
  "old_status",
  "new_status",
  "created_at"
],
"comms.announcements": [
  "id",
  "author_id",
  "title",
  "body_md",
  "pinned",
  "created_at"
],
"comms.comments": [
  "id",
  "entity_type project|task",
  "entity_id",
  "author_id",
  "body",
  "created_at"
],
"comms.notifications": [
  "id",
  "user_id",
  "type",
  "payload",
  "read_at"
],
"mail.routing_rules": [
  "id",
  "event_type",
  "recipient_type role|user|email",
  "recipient_value",
  "enabled"
],
"mail.outbox": [

```

```

        "id",
        "event_type",
        "payload_jsonb",
        "to_emails[]",
        "status pending|sent|failed",
        "attempts",
        "last_error",
        "created_at",
        "sent_at"
    ],
    "audit.audit_log": [
        "id",
        "actor_id",
        "actor_role",
        "action",
        "entity",
        "entity_id",
        "before_jsonb",
        "after_jsonb",
        "at",
        "INSERT-only trigger"
    ],
    "settings": [
        "key",

        "value: office_start_time=09:30, eod_lock=23:59, tz=Asia/Kolkata"
    ]
},
"mail_events": {
    "punch.in": {
        "default_to": "role:md",
        "body_includes": [
            "employee",
            "claimed_at",
            "server_at",
            "gap",
            "work_mode",
            "device_type",
            "os",
            "browser",
            "ip"
        ]
    },
    "punch.out": {
        "default_to": "role:md",
        "body_includes": "same as punch.in"
    },
    "leave.requested": {
        "default_to": "role:md"
    },
    "leave.decided": {
        "default_to": "user:requester"
    },
    "eod.missing": {
        "default_to": "role:md",
        "schedule": "20:30 IST"
    }
},
"schedule": [
    {
        "day": 1,
        "title": "Scaffold & Infrastructure",
        "verify": "compose up -> web :3000, /health 200, mailhog :8025, CI green"
    },
    {
        "day": 2,
        "title": "Auth Foundation",
        "verify": "seeded admin login, /auth/me, 6th bad login 429, httpOnly cookies"
    },
    {

```

```

    "day": 3,
    "title": "RBAC Engine + Audit Middleware",
    "verify": "custom role works live; 403s correct; writes in audit_log"
  },
  {
    "day": 4,
    "title": "Attendance Engine (Immutable Punches + Device Capture)",
    "verify": "dual times + device/os/browser/ip stored; no edit path for employee"
  },
  {
    "day": 5,
    "title": "Calendar Engine + Auto Day-Marking",
    "verify": "no-punch working day -> Absent; holiday overrides on re-run"
  },
  {
    "day": 6,
    "title": "Leave Management",
    "verify": "apply -> approve -> calendar Leave + balance used"
  },
  {
    "day": 7,
    "title": "EOD with Templates + Midnight Lock",
    "verify": "template prefill submit <1 min; next-day edit 403"
  },
  {
    "day": 8,
    "title": "Project Management",
    "verify": "assign task; assignee-only status update; timeline rows"
  },
  {
    "day": 9,
    "title": "Mail Engine (Zoho) + Admin Routing",
    "verify": "punch -> MD mail with dual times + device line; reroute works"
  },
  {
    "day": 10,
    "title": "Employee Dashboard",
    "verify": "punch, EOD, task, leave all from /dashboard; own activity log"
  },
  {
    "day": 11,
    "title": "MD Dashboard",
    "verify": "feed shows gap + device/os/ip; drill-down flags new device/IP"
  },
  {
    "day": 12,
    "title": "Admin Dashboard",
    "verify": "role built in UI works live; punch edit diff in audit"
  },
  {
    "day": 13,
    "title": "Communication, Bell & CSV Export",
    "verify": "announcement + bell; CSV with device/os/ip columns"
  },
  {
    "day": 14,
    "title": "Hardening, Seed, Docs & Ship",
    "verify": "clean clone -> compose up -> full demo <5 min; make smoke green"
  }
],
"known_limits": [
  "laptop vs desktop tower not distinguishable from browser user-agent (both = desktop-class)",
  "user-agent is spoofable by technical users; treat device as signal, not proof",
  "zoho mail daily send caps on free/standard tiers; monitor outbox failed count; digest toggle is a
future option"
]
}
===EMS_SPEC_JSON_END===

```